

UNIVERSIDAD DE GRANADA

PERIFÉRICOS Y DISPOSITIVOS DE INTERFAZ HUMANA

CURSO 2025/2026

Práctica 5. Experimentación con el sistema de salida de sonido

Autora:
Inés Ruiz Sánchez

19 de mayo de 2026



UNIVERSIDAD
DE GRANADA

Índice

1. Introducción y objetivos	2
2. Requisitos mínimos	2
2.1. Ejercicio 1	2
2.2. Ejercicio 2	4
2.3. Ejercicio 3	7
2.4. Ejercicio 4	8
2.5. Ejercicio 5	9
2.6. Ejercicio 6	10
3. Requisitos ampliados	12
3.1. Ejercicio 7	12
3.2. Ejercicio 8	14
3.2.1. Comparativa ejercicio 7 y 8.	17
4. Anexo	18
4.1. Fichero con requisitos mínimos	18
4.2. Fichero con requisitos ampliados	19

Índice de figuras

1. Convertir texto a sonido	2
2. Convertir texto a sonido	3
3. Descargar sonido	3
4. Fichero de sonido con apellido	4
5. Cargar y leer los ficheros de sonido	5
6. Cargar y leer los ficheros de sonido	6
7. Dibujo de fichero “nombre.wav”	6
8. Dibujo de fichero “apellidos.wav”	7
9. Cabeceras de los ficheros	8
10. Unión de dos sonidos	9
11. Gráfica de la onda unida	10
12. Creación de “basico.wav”	11
13. Filtro de frecuencia	12
14. Información del <i>filtro</i>	13
15. Gráfica de la onda con filtro	13
16. Efecto de eco	14
17. Gráfica del sonido con eco	15
18. Información de sonido “alreves”	16
19. Gráfica de onda “alreves”	17

1. Introducción y objetivos

Los objetivos concretos de esta práctica son:

- Identificar y representar gráficamente la forma de onda de señales de sonido.
- Conocer la estructura de un fichero típico de sonido (ficheros WAV).
- Entender y operar con los parámetros principales de una señal de sonido.

En el anexo⁴ se incluyen los ficheros de códigos al completo.

Para obtener los ficheros WAV he usado la página <https://speechgen.io/>.

Repositorio en GitHub: <https://github.com/ruiz314/PDIH/tree/main/P5>.

Para poder trabajar con R y RStudio en mi ordenador personal realicé los seminarios y tutoriales proporcionados por el profesor. Se encuentran en el siguiente repositorio: <https://github.com/ruiz314/PDIH/tree/main/S-sonido>.

2. Requisitos mínimos

Para superar esta práctica se proponen los siguientes ejercicios, que se podrán realizar en un mismo programa:

- Crear dos ficheros de sonido (WAV) para realizar los siguientes ejercicios. En el primero debe escucharse el nombre de la persona que realiza la práctica. En el segundo debe escucharse el apellido.
- Leer los dos ficheros de sonido creados y dibujar la forma de onda de ambos sonidos (por separado).
- Obtener la información de las cabeceras de ambos sonidos.
- Unir ambos sonidos en uno nuevo para escuchar el nombre y apellido correctamente.
- Dibujar la forma de onda de la señal y reproducir el sonido resultante (una vez unidos).
- Almacenar el sonido resultante en un archivo nuevo llamado “basico.wav”

2.1. Ejercicio 1

Para crear los ficheros de sonido en la página <https://speechgen.io/> hay que seleccionar el idioma y la voz (arriba a la izquierda). A continuación hay que escribir el texto en el recuadro. Observar Figura 1.

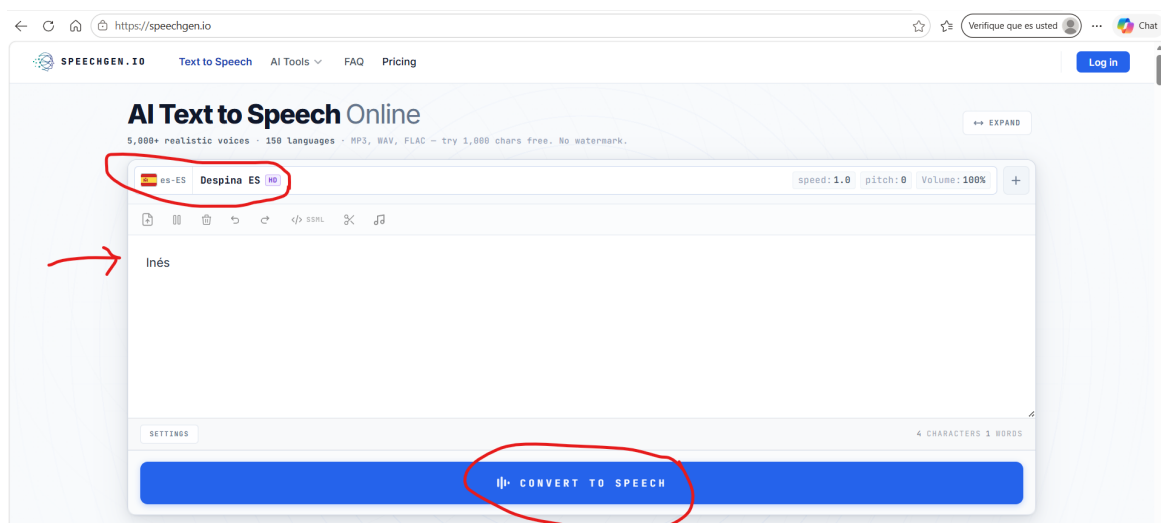


Figura 1: Convertir texto a sonido

Abajo a la izquierda hay un botón de “SETTINGS” (Figura 2) que sirve para cambiar la configuración del archivo.

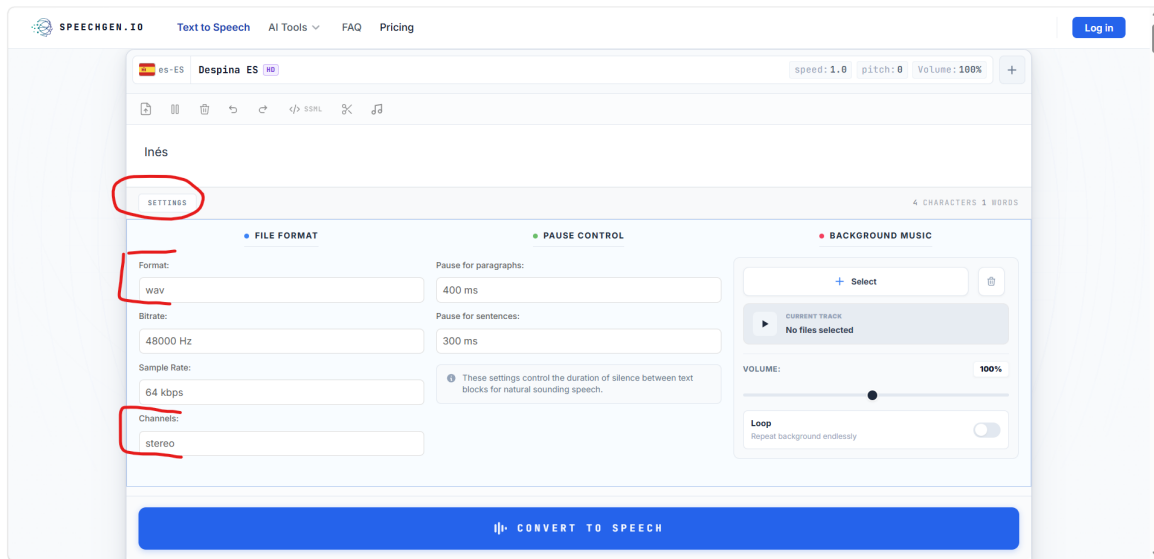


Figura 2: Convertir texto a sonido

Seleccionar formato “wav”.

Por último clicar en “CONVERT TO SPEECH”. Aparecerá abajo el audio, y podemos descargarlo pulsando el botón “DOWNLOAD”. Observar Figura 3.

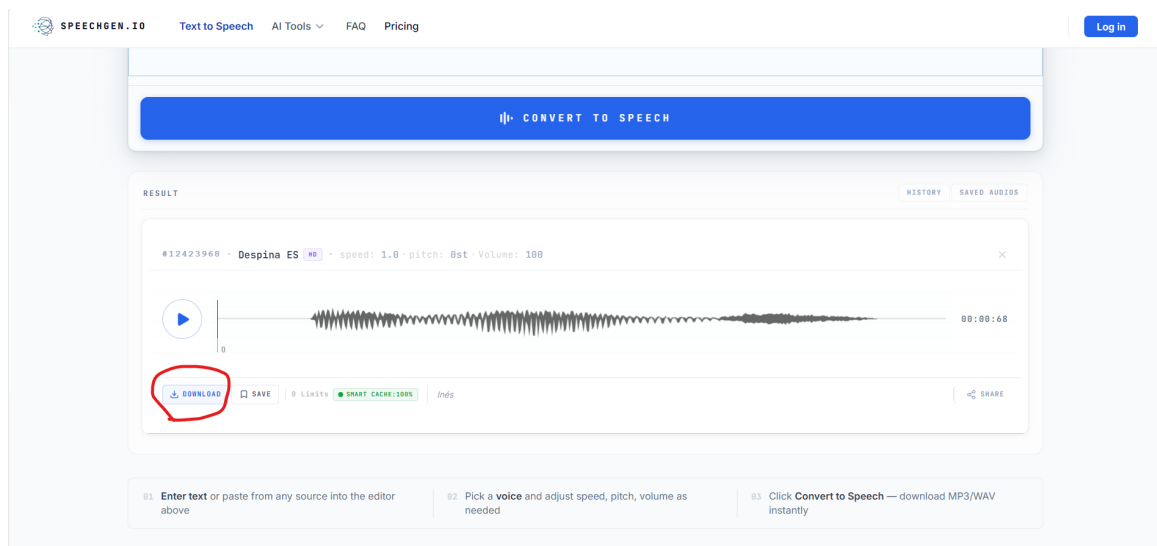


Figura 3: Descargar sonido

Para crear el fichero de sonido con los apellidos hay que realizar el mismo procedimiento. Observar Figura 4.

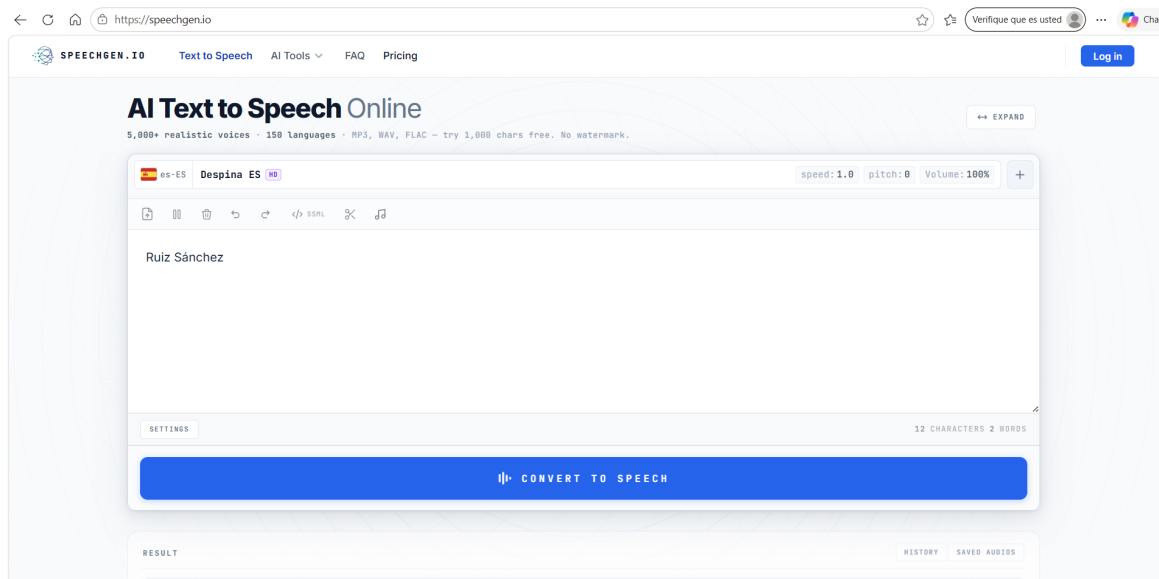


Figura 4: Fichero de sonido con apellido

2.2. Ejercicio 2

Para cargar los ficheros de audio se usa la función `readWave()`:

```

1 # Audio con el nombre
2 nombre <- readWave('nombre.wav')
3 nombre
4
5 # Audio con los apellidos
6 apellidos <- readWave('apellidos.wav')
7 apellidos

```

Listing 1: Cargar los ficheros de sonido en R

Llamar al fichero de sonido (líneas 3 y 7) hace que se muestre información como el número de muestras, la duración y el número de canales. Observar Figura 5.

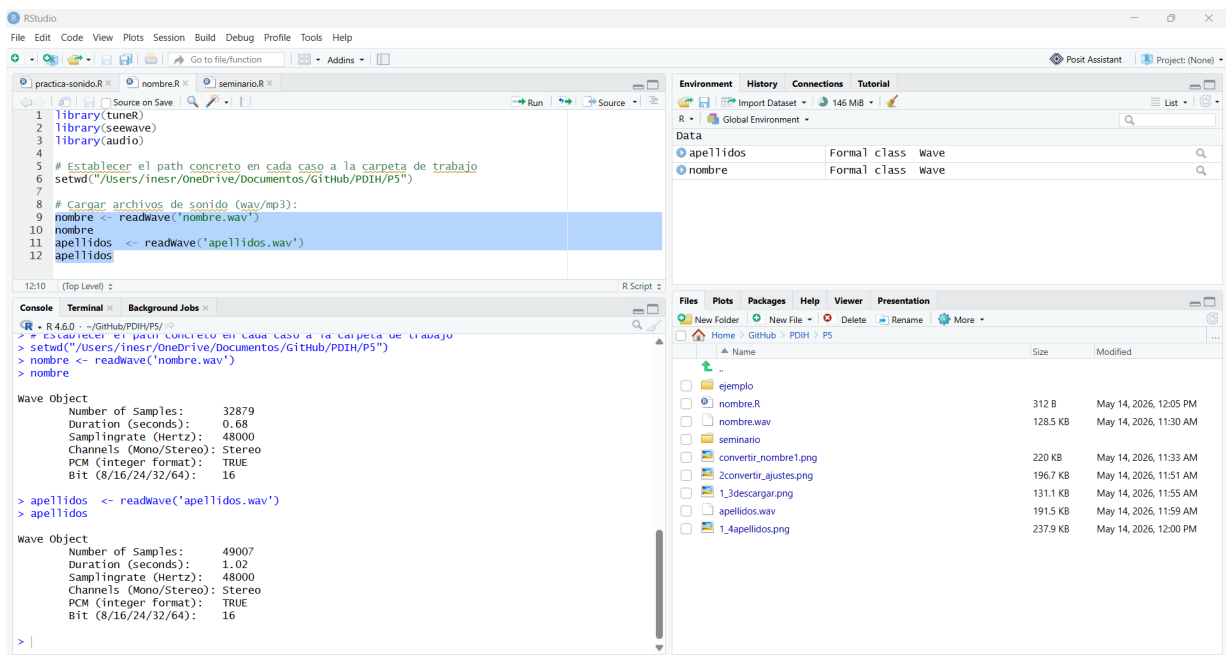


Figura 5: Cargar y leer los ficheros de sonido

Los datos obtenidos son los siguientes (ver Cuadro 1):

Cuadro 1: Información de los objetos Wave: *nombre* y *apellidos*

Parámetro	Objeto: nombre	Objeto: apellidos
Número de muestras	32879	49007
Duración (segundos)	0.68	1.02
Tasa de muestreo (Hz)	48000	48000
Canales	Stereo	Stereo
Formato PCM	TRUE	TRUE
Bit	16	16

Para obtener el dibujo de la onda la función `plot` necesita saber el número de muestras que tiene el fichero. Observamos que el fichero *nombre* tiene 32879 muestras y el fichero *apellidos* tiene 49007.

```

1 # Mostrar la onda del sonido
2 plot( extractWave(nombre, from = 1, to = 32879) )
3
4 plot( extractWave(apellidos, from = 1, to = 49007) )

```

Listing 2: Comandos para dibujar las ondas

Al usar la función `plot` obtenemos la gráfica que se muestra en la Figura 6:

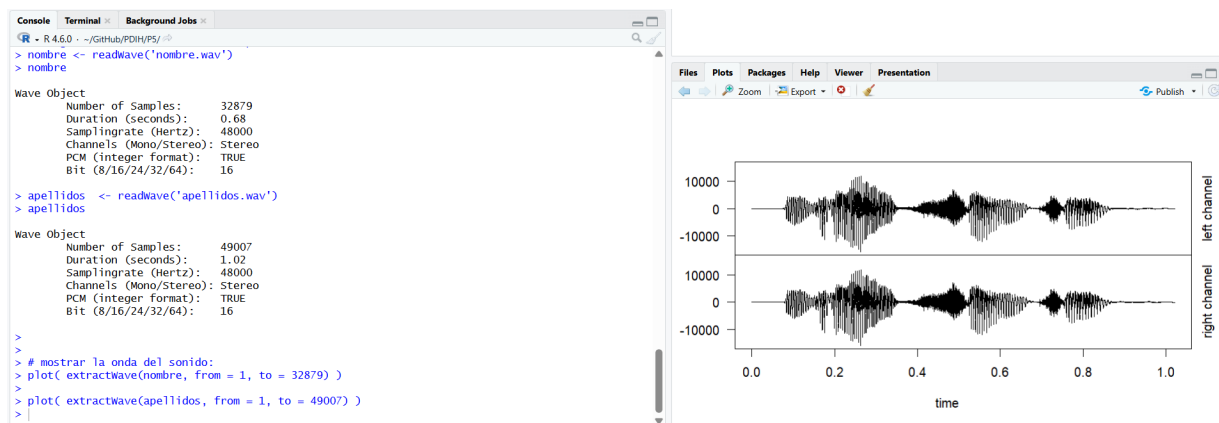


Figura 6: Cargar y leer los ficheros de sonido

El dibujo de la onda del archivo de sonido “nombre.wav” es el que se muestra en la Figura 7. El dibujo obtenido para el sonido de “apellidos.wav” se observa en la Figura 8.

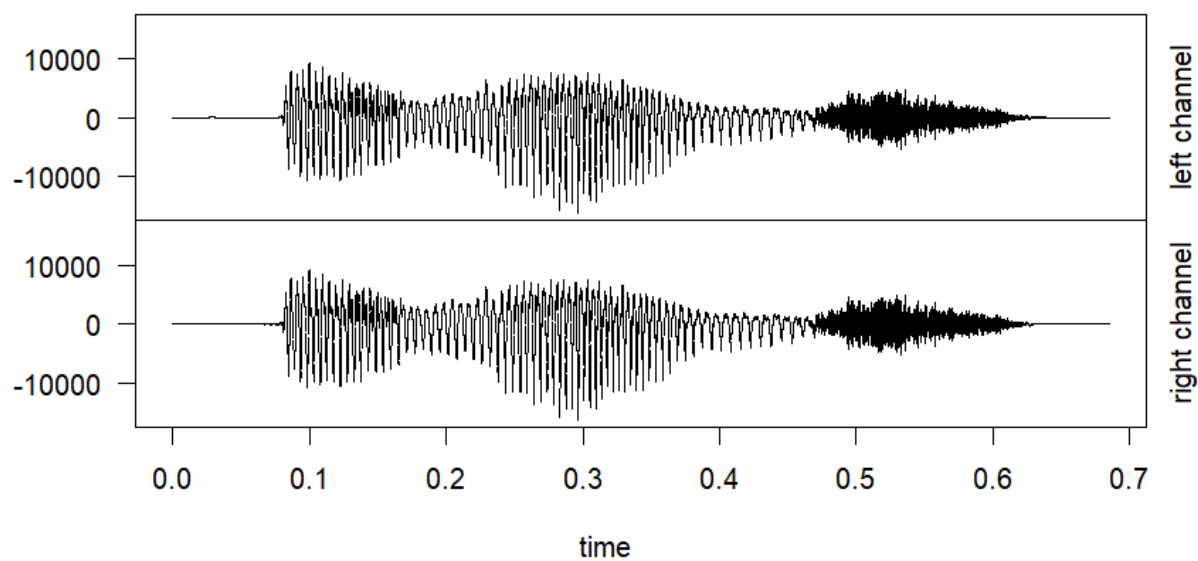


Figura 7: Dibujo de fichero “nombre.wav”

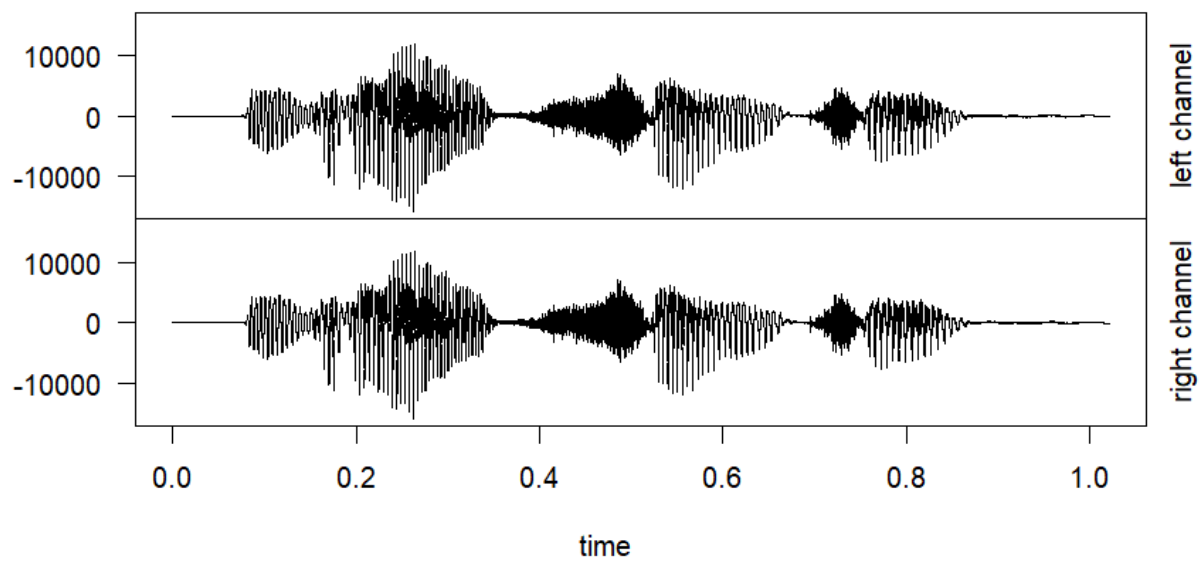


Figura 8: Dibujo de fichero “apellidos.wav”

2.3. Ejercicio 3

Usando la función `str()` se puede inspeccionar las cabeceras de archivos de sonido.

```
1 str(nombre)  
2 str(apellidos)
```

Listing 3: Funciones para las cabeceras de los ficheros de sonido

En la siguiente imagen se ve la información obtenida (Figura 9).


```

10 # onda de ambos sonidos (por separado).
11
12 # Cargar archivos de sonido (wav):
13 nombre <- readWave('nombre.wav')
14 nombre
15 apellidos <- readWave('apellidos.wav')
16 apellidos
17
18
19 # Mostrar la onda del sonido:
20 plot( extractWave(nombre, from = 1, to = 32879) )
21 plot( extractWave(apellidos, from = 1, to = 49007) )
22
23 # Ejercicio 3. Obtener la información de las cabeceras de ambos sonidos.
24 str(nombre)
25 str(apellidos)
26
27

```

```

> plot( extractWave(nombre, from = 1, to = 32879) )
> plot( extractWave(apellidos, from = 1, to = 49007) )
> str(nombre)
Formal class 'Wave' [package "tuner"] with 6 slots
..@ left      : int [1:32879] 0 0 0 0 0 0 0 0 0 0 ...
..@ right     : int [1:32879] 0 0 0 0 0 0 0 0 0 0 ...
..@ stereo    : logi TRUE
..@ samp.rate : int 48000
..@ bit       : int 16
..@ pcm       : logi TRUE
> str(apellidos)
Formal class 'Wave' [package "tuner"] with 6 slots
..@ left      : int [1:49007] 0 0 0 0 0 0 0 0 0 0 ...
..@ right     : int [1:49007] 0 0 0 0 0 0 0 0 0 0 ...
..@ stereo    : logi TRUE
..@ samp.rate : int 48000
..@ bit       : int 16
..@ pcm       : logi TRUE
>

```

Figura 9: Cabeceras de los ficheros

Como ambos sonidos tienen dos canales, el atributo *left* y el atributo *right* valen lo mismo para cada sonido. Además, confirmamos que tienen dos canales porque la variable *stereo* es verdadera.

2.4. Ejercicio 4

Con la función `pastew()` se unen dos sonidos. Hay que prestar atención al orden, pues el primer argumento será el segundo sonido que se escuche y el segundo argumento será el primer sonido que se escuche.

```
1 union <- pastew(apellidos, nombre, output="Wave")
```

Listing 4: Función para unir dos sonidos

En la Figura 10 se puede observar la información del sonido.

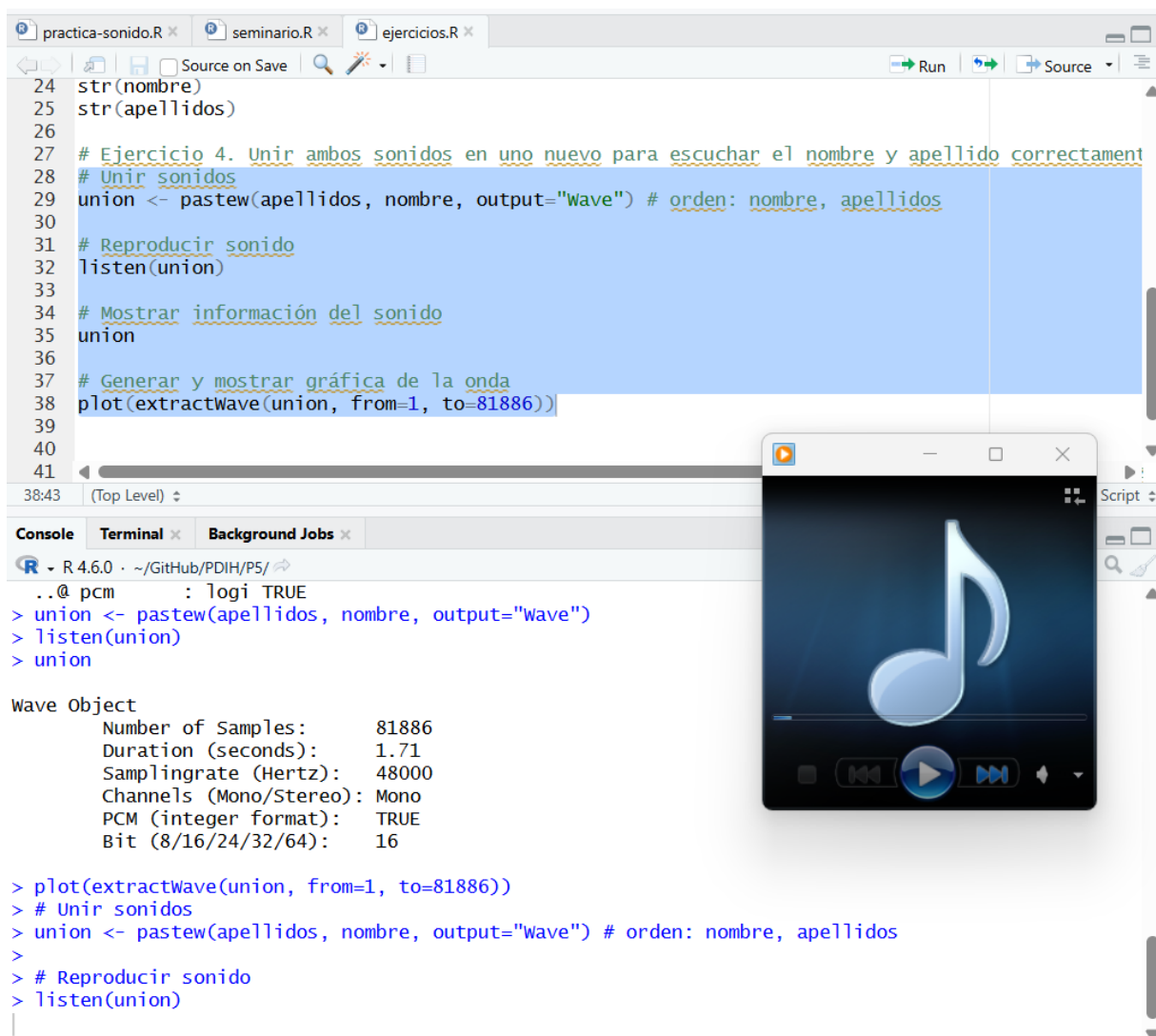


Figura 10: Unión de dos sonidos

En el Cuadro 2 se pueden observar los datos de los tres ficheros. La duración del fichero *union* es la suma de las duraciones del fichero *nombre* y el fichero *apellidos*, aproximadamente. Sucede lo mismo con el número de muestras: $32,879 + 49,007 = 81,886$.

Cuadro 2: Comparativa de los objetos Wave: *nombre*, *apellidos* y *union*

Parámetro	Objeto: nombre	Objeto: apellidos	Objeto: union
Número de muestras	32879	49007	81886
Duración (segundos)	0.68	1.02	1.71
Tasa de muestreo (Hz)	48000	48000	48000
Canales	Stereo	Stereo	Mono
Formato PCM	TRUE	TRUE	TRUE
Bit	16	16	16

2.5. Ejercicio 5

Llamando a la función *plot()* se genera una imagen con la onda resultante (Observar la Figura 11).

```

1 # Generar y mostrar dibujo de la onda
2 plot(extractWave(union, from=1, to=81886))
3

```

```

4 # Reproducir sonido
5 listen(union)

```

Listing 5: Generación de gráfica de unión

Además, para reproducir el sonido nuevo generado se usa la función `listen(union)` y abre el reproductor en el ordenador. Se puede observar cómo se abre en la Figura 10.

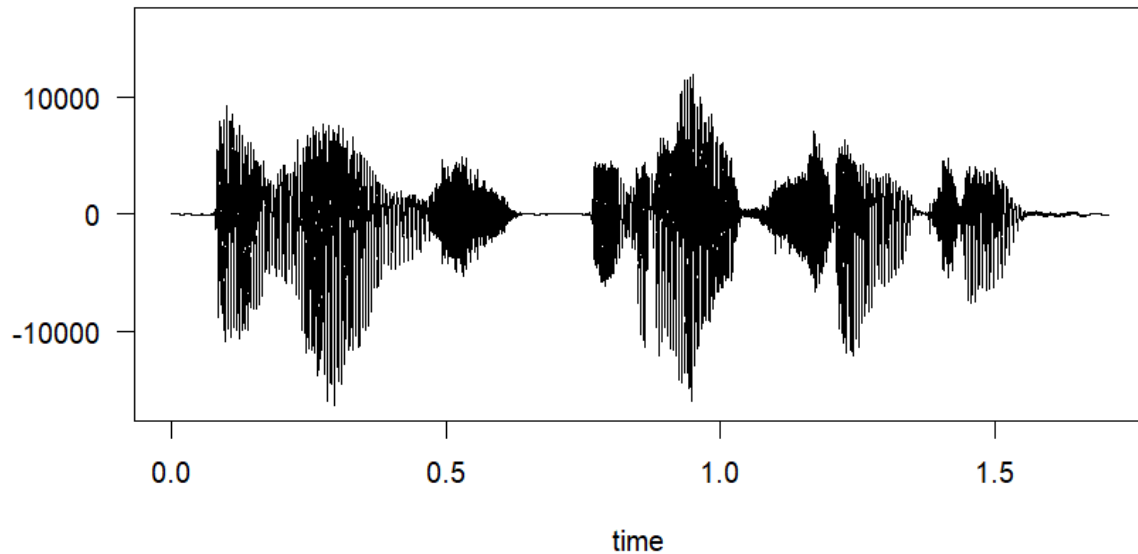


Figura 11: Gráfica de la onda unida

2.6. Ejercicio 6

Para guardar en disco el nuevo sonido generado se llama a la función `writeWave()` indicándole el sonido, y el nombre del fichero:

```

1 writeWave(union, file.path("basico.wav"))
2 basico <- readWave('basico.wav')
3 basico

```

Listing 6: Almacenar sonido union en fichero “basico.wav”

A continuación, en la Figura 12 se puede observar que la llamada a la función no genera ningún error y que el sonido ha sido almacenado con éxito. Además, se muestran los datos de la onda y coinciden con los datos de la onda *union*.

The screenshot displays the R Studio environment. The top pane shows an R script with several exercises. The bottom pane shows the console output for the last two lines of the script.

```
23 # Ejercicio 3. Obtener la información de las cabeceras de ambos sonidos.  
24 str(nombre)  
25 str(apellidos)  
26  
27 # Ejercicio 4. Unir ambos sonidos en uno nuevo para escuchar el nombre y apellido correctamente  
28 # Unir sonidos  
29 union <- paste0(apellidos, nombre, output="wave") # orden: nombre, apellidos  
30  
31 # Mostrar información del sonido  
32 union  
33  
34 # Ejercicio 5. Dibujar la forma de onda de la señal y reproducir el sonido resultante (una vez)  
35 # Generar y mostrar dibujo de la onda  
36 plot(extractWave(union, from=1, to=81886))  
37  
38 # Reproducir sonido  
39 listen(union)  
40  
41 # Ejercicio 6. Almacenar el sonido resultante en un archivo nuevo llamado "basico.wav"  
42 writeWave(union, file.path("basico.wav"))  
43 basico <- readWave('basico.wav')  
44 basico  
45
```

Console Output:

```
> writeWave(union, file.path("basico.wav"))  
> basico <- readWave('basico.wav')  
> basico  
  
Wave Object  
  Number of Samples: 81886  
  Duration (seconds): 1.71  
  Samplingrate (Hertz): 48000  
  Channels (Mono/Stereo): Mono  
  PCM (integer format): TRUE  
  Bit (8/16/24/32/64): 16  
  
> |
```

Figura 12: Creación de “basico.wav”

3. Requisitos ampliados

Como requisitos ampliados (opcionales para subir nota) se propone:

- Pasarle un filtro de frecuencia para eliminar las frecuencias entre 10.000Hz y 20.000Hz. Almacenar la señal obtenida como un fichero WAV denominado “filtrado.wav”
- Tomar el sonido que se creó antes (lo tendremos en el archivo llamado “basico.wav”) para aplicarle el efecto de eco. Guardar ese sonido en un archivo nuevo llamado “eco.wav”. A continuación, se le debe dar la vuelta al sonido y almacenarlo como un fichero llamado “alreves.wav”

3.1. Ejercicio 7

La función `bwfilter()` aplica un filtro de frecuencia, eliminando las muestras entre las frecuencias (en Hz) indicadas.

```
1 filtro <- bwfilter(basico, f=basico@samp.rate, from=10000, to=20000, bandpass= FALSE,
2   output = "Wave")
3 writeWave(filtro, file.path("filtrado.wav"))
```

Listing 7: Filtro de frecuencia

Al poner `f = basico@samp.rate` la función lee automáticamente la cabecera del archivo y aplica el filtro de frecuencias de manera exacta y sin errores.

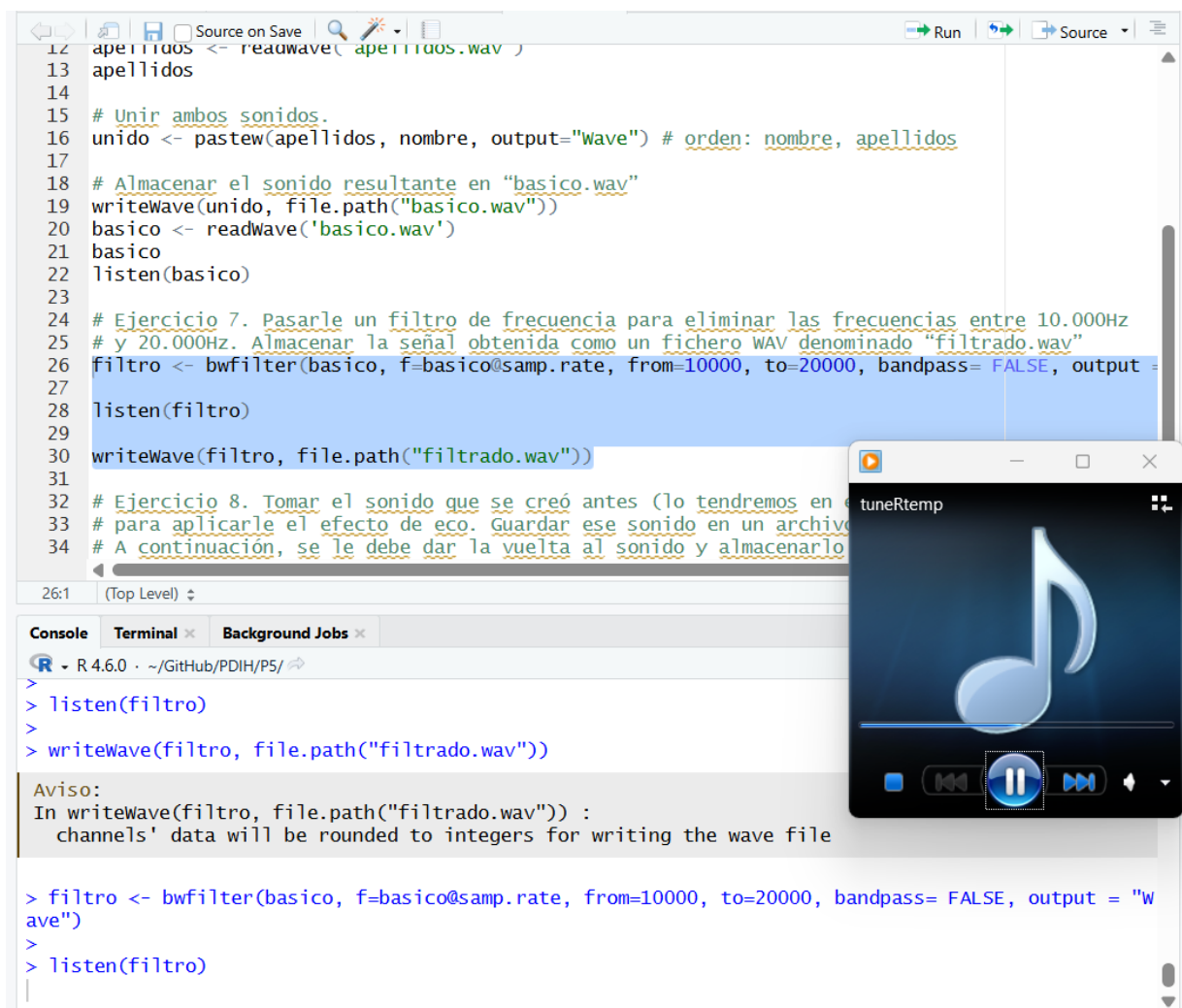


Figura 13: Filtro de frecuencia

En la Figura 13 da un aviso al ejecutar la función `writeWave`. Este aviso no afecta al ejercicio ya que el redondeo no es perceptible para el oído humano.

Para ver información sobre el sonido uso la función `str` y la propia llamada al sonido. Se puede observar en *Listing 8* y *Figura 14*.

```
1 str(filtro)
2 filtro
```

Listing 8: Información del filtro de frecuencia

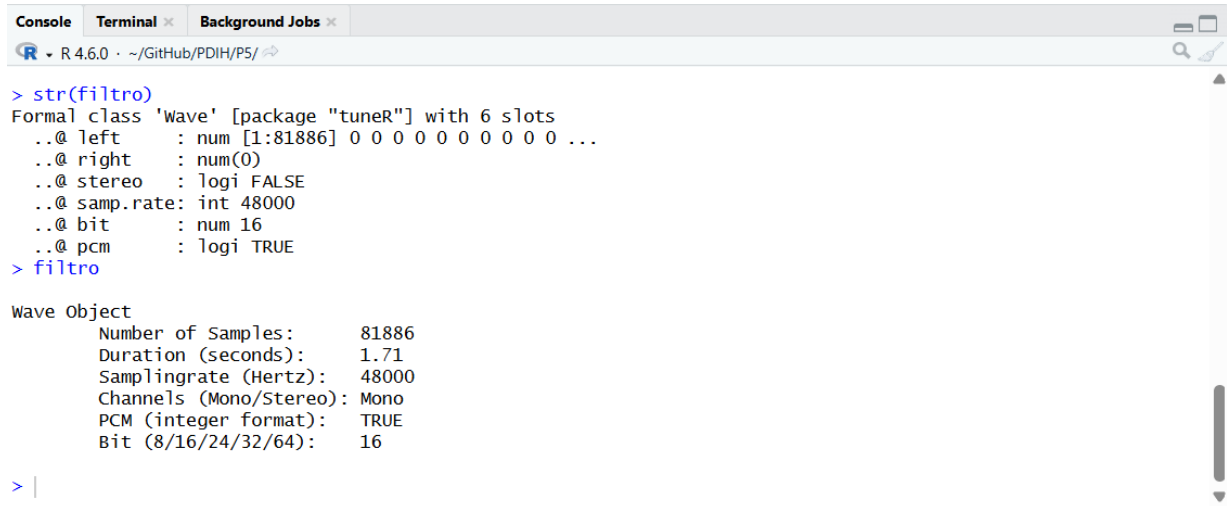


Figura 14: Información del *filtro*

Con esa información obtenida se puede usar la función `plot` para dibujar la onda. Ver Figura 15.

```
1 plot(extractWave(filtro, from = 1, to = 81886))
```

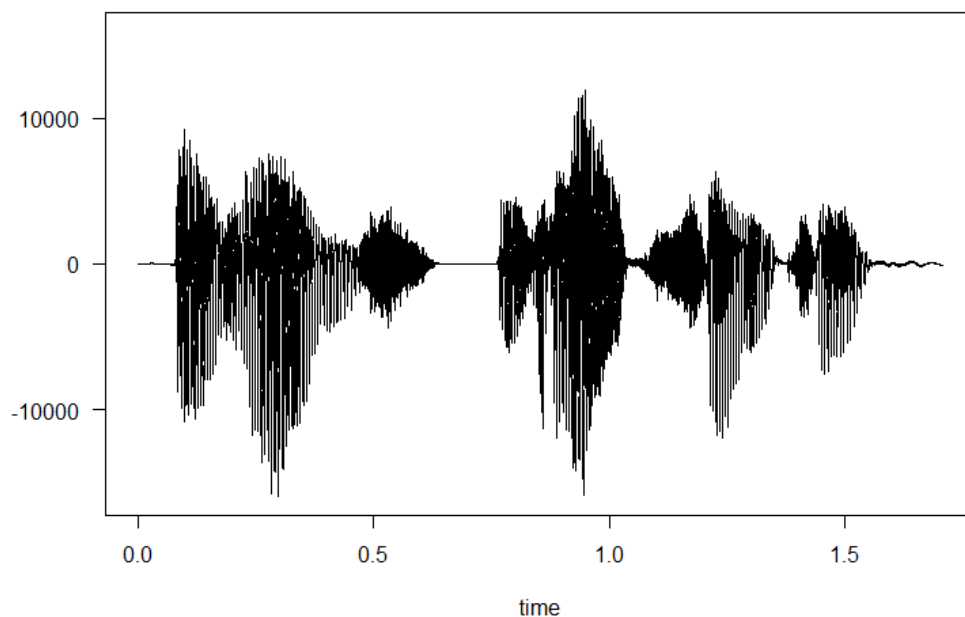


Figura 15: Gráfica de la onda con filtro

3.2. Ejercicio 8

Con la función `echo` se le puede aplicar el efecto de eco, donde `delay` es el retraso del eco, y `amp` es la amplitud/volumen del eco.

```
1 # Generación de eco
2 filtroECO <- echo(basico, f=basico@samp.rate, amp=c(0.8,0.4,0.2), delay=c(1,2,3),
3   output="Wave")
4
5 # Escuchar audio
6 listen(filtroECO)
7
8 # Mostrar información del sonido
9 str(filtroECO)
10 filtroECO
```

Listing 9: Aplicación de eco

En la Figura 16 se puede observar que la llamada a la función `echo` no genera ningún error. Además, se ve la información de la onda gracias a la función `str` y a la llamada de la propia onda. La duración del sonido es de 4,71 segundos, frente a los 1,71 segundos que duraba el sonido “basico”.

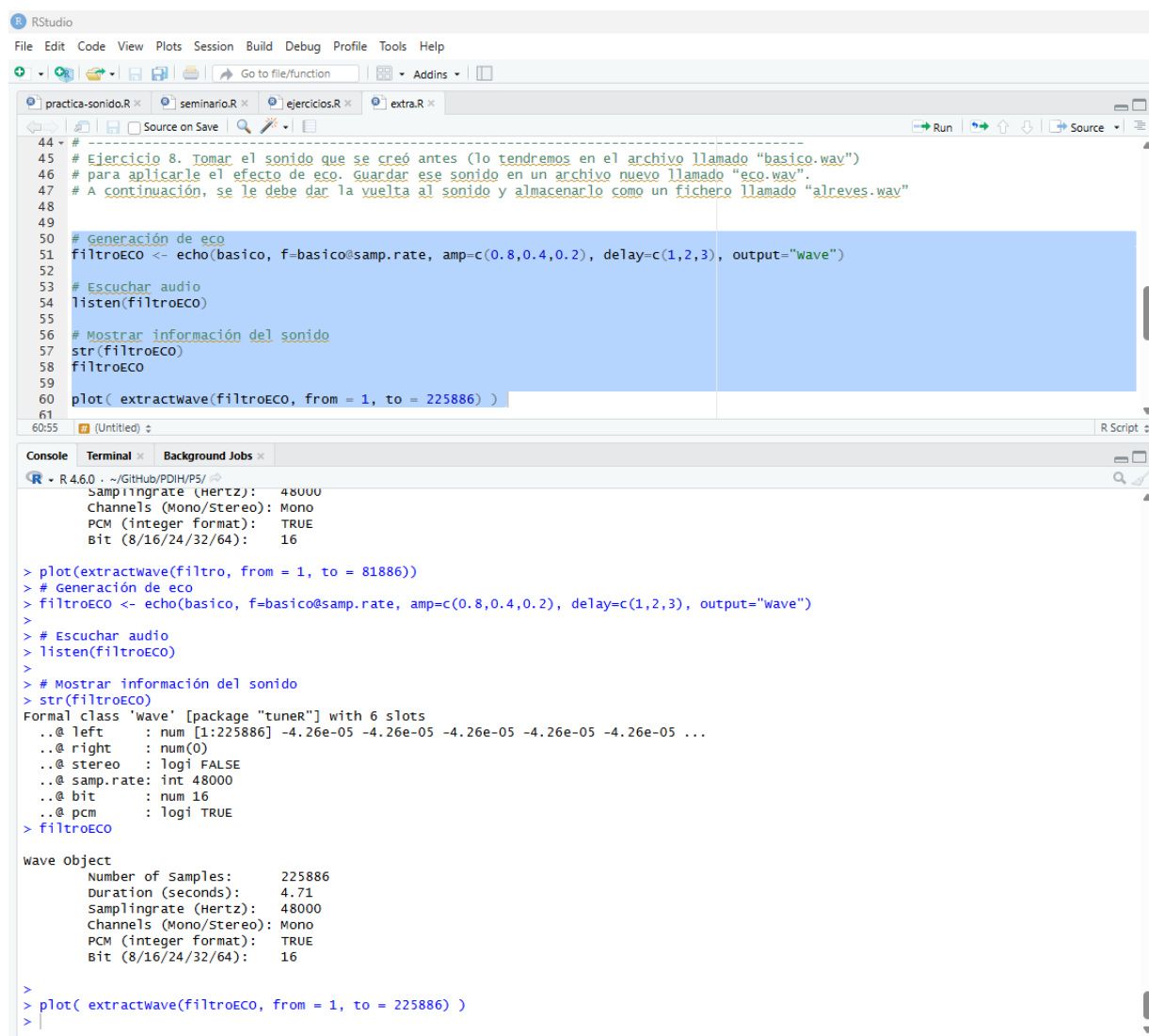


Figura 16: Efecto de eco

Con la llamada a la función `plot` se puede obtener la gráfica de la onda. Se muestra en la Figura 17.

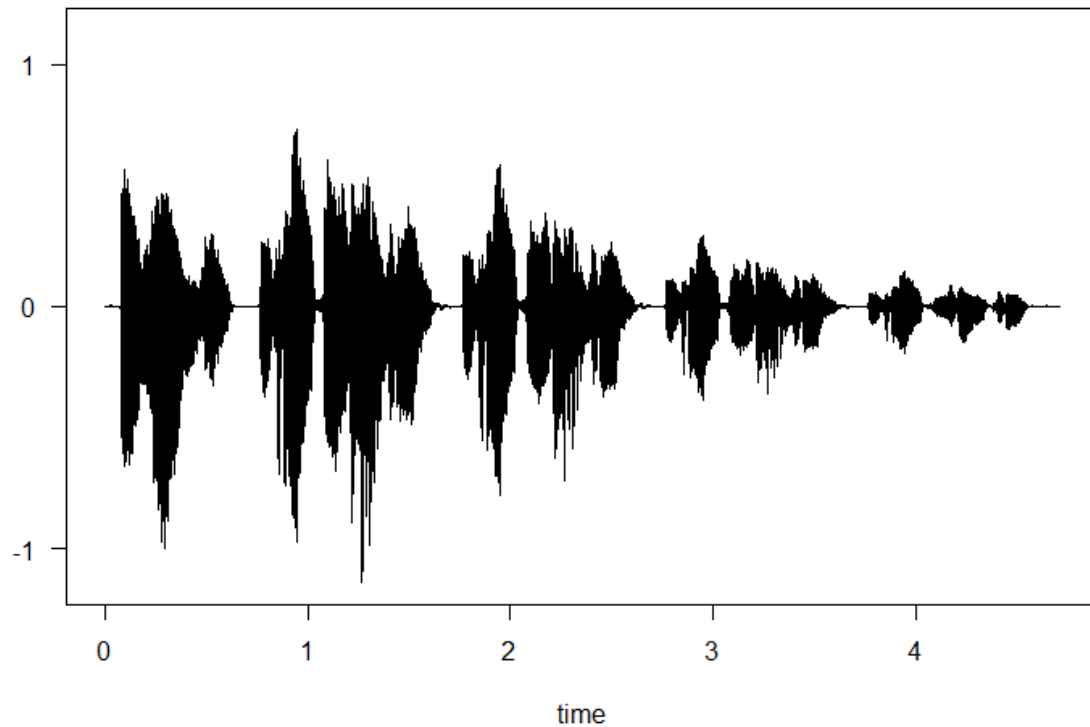


Figura 17: Gráfica del sonido con eco

Para guardar el sonido nuevo generado en un archivo se usa la función `writeWave` con los parámetros que se muestran en *Listing 10*.

```
1 writeWave(filtroECO, file.path("eco.wav") )
```

Listing 10: Guardar sonido con eco

Por último, para darle la vuelta se usa la función `revw` como se observa en *Listing 11*. Además, se usa la función `writeWave` para almacenarla en un fichero.

```
1 # Dar la vuelta
2 alreves <- revw(filtroECO, output="Wave")
3
4 # Escuchar sonido
5 listen(alreves)
6
7 # Guardar en "alreves.wav"
8 writeWave(alreves, file.path("alreves.wav") )
9
10 # Mostrar información
11 str(alreves)
12 alreves
13
14 # Generar gráfica de sonido
15 plot( extractWave(alreves, from = 1, to = 225886) )
```

Listing 11: Dar la vuelta al sonido

En la Figura 18 se puede observar la información del sonido gracias a la función `str`. De nuevo, el archivo tiene una duración de 4,71 segundos, al igual que el sonido “filtroECO”.

```

65 # Dar la vuelta
66 alreves <- revw(filtroECO, output="wave")
67
68 # Escuchar sonido
69 listen(alreves)
70
71 # Guardar en "alreves.wav"
72 writewave(alreves, file.path("alreves.wav"))
73
74 # Mostrar información
75 str(alreves)
76 alreves
77
78 # Generar gráfica de sonido
79 plot(extractwave(alreves, from = 1, to = 225886))

```

```

> alreves <- revw(filtroECO, output="wave")
>
> # Escuchar sonido
> listen(alreves)
>
> # Guardar en "alreves.wav"
> writewave(alreves, file.path("alreves.wav"))

```

Aviso:
In writewave(alreves, file.path("alreves.wav")) :
channels' data will be rounded to integers for writing the wave file

```

>
> # Mostrar información
> str(alreves)
Formal class 'wave' [package "tuner"] with 6 slots
..@ left      : num [1:225886] -4.26e-05 -9.75e-04 -9.75e-04 -9.75e-04 ...
..@ right     : num(0)
..@ stereo    : logi FALSE
..@ samp.rate: int 48000
..@ bit       : num 16
..@ pcm       : logi TRUE
> alreves

```

```

Wave object
  Number of samples: 225886
  Duration (seconds): 4.71
  Samplingrate (Hertz): 48000
  Channels (Mono/Stereo): Mono
  PCM (integer format): TRUE
  Bit (8/16/24/32/64): 16

```

```

>
> # Generar gráfica de sonido
> plot(extractwave(alreves, from = 1, to = 225886))
>

```

Figura 18: Información de sonido “alreves”

La Figura 19 muestra la gráfica de la onda.

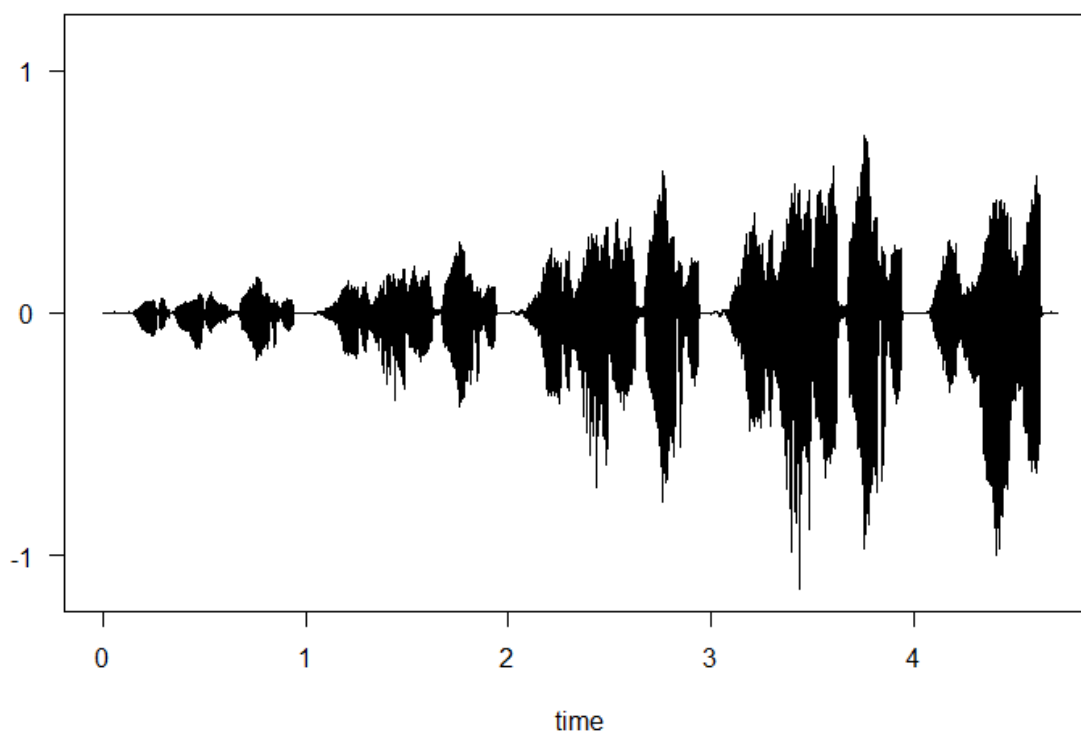


Figura 19: Gráfica de onda “alreves”

3.2.1. Comparativa ejercicio 7 y 8.

Con los datos obtenidos de las ondas, se muestra una tabla (Cuadro 3) a modo de resumen.

Cuadro 3: Comparativa de las propiedades de las ondas

Propiedad / Métrica	Audio Original (basico)	Audio Filtrado (filtro)	Audio con Eco (filtroECO)	Audio Invertido (alreves)
Número de Muestras	81886	81886	225886	225886
Duración Total	1.71 s	1.71 s	4.71 s	4.71 s
Canales	Mono	Mono	Mono	Mono
left	num [1:81886]	num [1:81886]	num [1:225886]	num [1:225886]
right	num(0)	num(0)	num(0)	num(0)

Al aplicar el **filtro de frecuencia** (`bwfilter`) para eliminar las frecuencias entre 10,000 Hz y 20,000 Hz, se observa que el número de muestras (81,886) y la duración (1,71 s) se mantienen idénticos al archivo básico. El filtro no altera sus atributos.

Al introducir el **efecto de eco** mediante la función `echo()`, se produce un incremento de las dimensiones del archivo. La señal pasa de 1,71 segundos a 4,71 segundos, lo que supone un aumento de 3,00 segundos. Esto se debe al `delay = c(1, 2, 3)`, ya que desplaza la onda original. El número total de muestras aumenta de 81,886 a 225,886.

La función `revw()` no altera los parámetros de la onda. Solo cambia el orden de almacenamiento de datos en el búfer. El búfer comienza con la última muestra grabada y termina por la primera. Físicamente la energía y duración son simétricas, modificando únicamente el sentido de la reproducción. Esto se puede observar en las gráficas de las ondas (Figura 17 para la onda `filtroECO` y Figura 19 para la onda `alreves`).

4. Anexo

4.1. Fichero con requisitos mínimos

Ejercicios del 1 al 6.

```
1 # PDIH. Práctica 5
2 library(tuneR)
3 library(seewave)
4 library(audio)
5
6 # Establecer el path concreto a la carpeta de trabajo
7 setwd("/Users/inesr/OneDrive/Documentos/GitHub/PDIH/P5")
8
9 # -----
10 # Ejercicio 2. Leer los dos ficheros de sonido creados y dibujar la forma de
11 # onda de ambos sonidos (por separado).
12
13 # Cargar archivos de sonido (wav):
14 nombre <- readWave('nombre.wav')
15 nombre
16 apellidos <- readWave('apellidos.wav')
17 apellidos
18
19 # Mostrar la onda del sonido:
20 plot( extractWave(nombre, from = 1, to = 32879) )
21 plot( extractWave(apellidos, from = 1, to = 49007) )
22
23 # -----
24 # Ejercicio 3. Obtener la información de las cabeceras de ambos sonidos.
25 str(nombre)
26 str(apellidos)
27
28 # -----
29 # Ejercicio 4. Unir ambos sonidos en uno nuevo para escuchar el nombre y apellido
30 # correctamente.
31 # Unir sonidos
32 union <- pasteW(apellidos, nombre, output="Wave") # orden: nombre, apellidos
33
34 # Mostrar información del sonido
35 union
36
37 # -----
38 # Ejercicio 5. Dibujar la forma de onda de la señal y reproducir el sonido
39 # resultante (una vez unidos).
40
41 # Generar y mostrar dibujo de la onda
42 plot(extractWave(union, from=1, to=81886))
43
44 # Reproducir sonido
45 listen(union)
46
47 # -----
48 # Ejercicio 6. Almacenar el sonido resultante en un archivo nuevo llamado
49 # "basico.wav"
50
51 # Guardar sonido en archivo
52 writeWave(union, file.path("basico.wav"))
53
54 basico <- readWave('basico.wav')
55 basico
```

4.2. Fichero con requisitos ampliados

Ejercicios 7 y 8.

```
1 # PDIH. Práctica 5. Ejercicios de ampliación
2 library(tuneR)
3 library(seewave)
4 library(audio)
5
6 # Establecer el path concreto a la carpeta de trabajo
7 setwd("/Users/inesr/OneDrive/Documentos/GitHub/PDIH/P5")
8
9 # Cargar archivos de sonido:
10 nombre <- readWave('nombre.wav')
11 nombre
12 apellidos <- readWave('apellidos.wav')
13 apellidos
14
15 # Unir ambos sonidos.
16 unido <- pastew(apellidos, nombre, output="Wave") # orden: nombre, apellidos
17
18 # Almacenar el sonido resultante en "basico.wav"
19 writeWave(unido, file.path("basico.wav"))
20 basico <- readWave('basico.wav')
21 basico
22 listen(basico)
23
24 # -----
25 # Ejercicio 7. Pasarle un filtro de frecuencia para eliminar las frecuencias entre
26 # 10.000Hz y 20.000Hz.
27 # Almacenar la señal obtenida como un fichero WAV denominado "filtrado.wav"
28
29 # Aplicar filtro
30 filtro <- bwfilter(basico, f=basico@samp.rate, from=10000, to=20000, bandpass= FALSE,
31   output = "Wave")
32
33 # Escuchar el resultado
34 listen(filtro)
35
36 # Almacenar la señal en archivo
37 writeWave(filtro, file.path("filtrado.wav"))
38
39 # Ver información del sonido
40 str(filtro)
41 filtro
42
43 # Obtener gráfica de la onda
44 plot(extractWave(filtro, from = 1, to = 81886))
45
46 # -----
47 # Ejercicio 8. Tomar el sonido que se creó antes (lo tendremos en el archivo llamado
48 # "basico.wav") para aplicarle el efecto de eco. Guardar ese sonido en un archivo
49 # nuevo llamado "eco.wav". A continuación, se le debe dar la vuelta al sonido y
50 # almacenarlo como un fichero llamado "alreves.wav"
51
52 # Generación de eco
53 filtroECO <- echo(basico, f=basico@samp.rate, amp=c(0.8,0.4,0.2), delay=c(1,2,3),
54   output="Wave")
55
56 # Escuchar audio
57 listen(filtroECO)
58
59 # Mostrar información del sonido
```

```

59 str(filtroECO)
60 filtroECO
61
62 plot( extractWave(filtroECO, from = 1, to = 225886) )
63
64 # Guardar en "eco.wav"
65 writeWave(filtroECO, file.path("eco.wav") )
66
67 # Dar la vuelta
68 alreves <- revw(filtroECO, output="Wave")
69
70 # Escuchar sonido
71 listen(alreves)
72
73 # Guardar en "alreves.wav"
74 writeWave(alreves, file.path("alreves.wav") )
75
76 # Mostrar información
77 str(alreves)
78 alreves
79
80 # Generar gráfica de sonido
81 plot( extractWave(alreves, from = 1, to = 225886) )

```